

# Distributed Hybrid Quantum-Classical Communication Between IITR and IITJ Using SimulaQron, ngrok, and ETSI-style QKD APIs

Jithin S

## 1 GitHub Repository

The complete updated project code is available at:

<https://github.com/jithinjithu10/qkd-network-emulation/tree/main/May11th>

The repository contains:

- Hybrid QKD framework
- SimulaQron BB84 implementation
- ETSI-style KMS APIs
- AES-256-GCM encryption layer
- Inter-KMS synchronization
- Metadata synchronization
- Replay protection
- Session management
- Distributed communication support
- ngrok internet tunneling support

The implementation represents a distributed hybrid quantum-classical secure communication framework using a simulated BB84 quantum layer. :contentReference[oaicite:0]index=0

## 2 Objective

The objective is to establish secure distributed communication between:

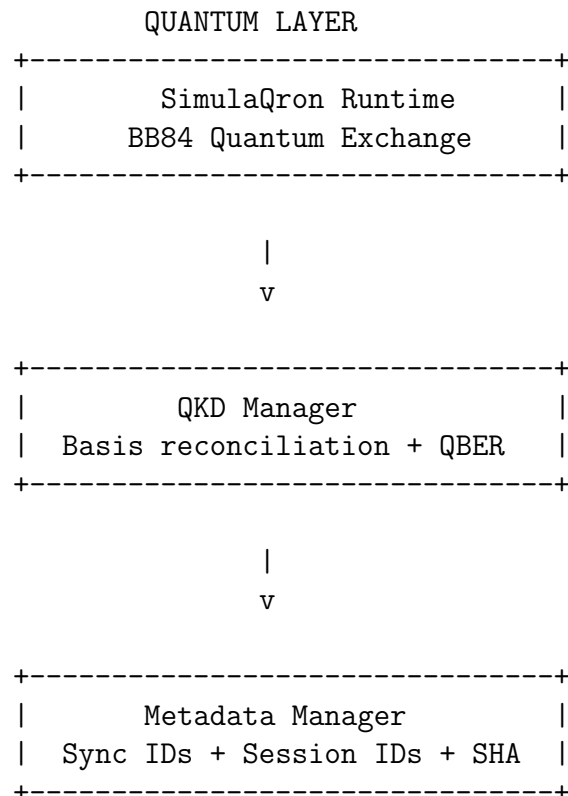
- IITR (Sender + Main KMS + BB84 Key Generator)
- IITJ (Receiver + Client KMS)

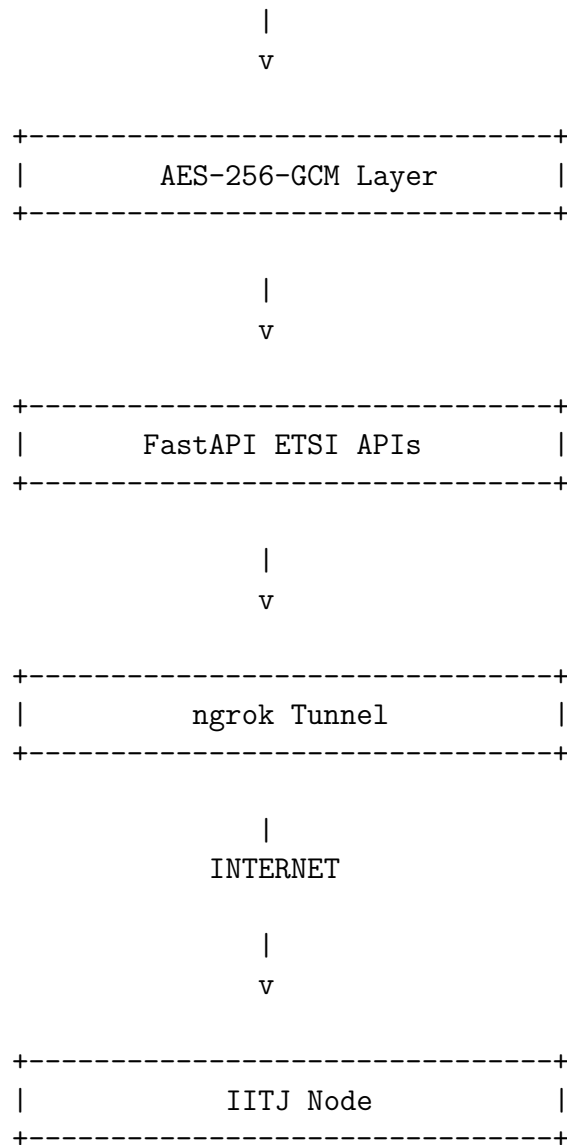
using:

- SimulaQron
- BB84 Quantum Key Distribution
- AES-256-GCM
- FastAPI
- ETSI-style APIs
- ngrok tunnels
- distributed synchronization
- replay protection

## 3 High-Level Architecture

### 3.1 Complete Architecture





## 4 Important Understanding

### 4.1 Quantum Channel

The quantum channel is simulated LOCALLY using SimulaQron.

Actual qubits are NOT transmitted through the internet.

SimulaQron simulates:

- qubits
- Hadamard gates
- quantum measurement
- BB84 encoding
- basis reconciliation

- quantum behavior

inside the local WSL Ubuntu runtime.

## 4.2 Classical Channel

The classical channel is REAL.

The following travel through the internet:

- encrypted ciphertext
- synchronization metadata
- session IDs
- timestamps
- SHA-256 hashes
- replay protection nonces
- acknowledgements

through:

FastAPI + ngrok + Internet

## 5 Machines

| Machine | Responsibility                                 |
|---------|------------------------------------------------|
| IITR    | Main KMS Server + BB84 Key Generator + Sender  |
| IITJ    | Client KMS + Receiver + Synchronization Client |

## 6 Step 1: Install WSL Ubuntu

On BOTH IITR and IITJ systems:

Open PowerShell as Administrator.

Run:

```
wsl --install
```

Restart the system.

Open Ubuntu.

Set:

- username
- password

## 7 Step 2: Clone the Repository

On BOTH IITR and IITJ:

Open Ubuntu terminal.

Run:

```
cd ~
```

```
git clone https://github.com/jithinjithu10/qkd-network-emulation.git
```

Go inside the project:

```
cd qkd-network-emulation/May11th
```

## 8 Step 3: Create Python Environment

On BOTH IITR and IITJ:

```
cd ~
```

```
mkdir qkd_demo
```

```
cd qkd_demo
```

```
python3 -m venv simulaqron_env
```

Activate:

```
source simulaqron_env/bin/activate
```

## 9 Step 4: Install Dependencies

Install required packages:

```
pip install simulaqron
```

```
pip install cqcl
```

```
pip install fastapi
```

```
pip install uvicorn
```

```
pip install requests
```

```
pip install cryptography
```

## 10 Step 5: Configure SimulaQron

On BOTH IITR and IITJ:

```
source ~/qkd_demo/simulaqron_env/bin/activate
```

Set default nodes:

```
simulaqron nodes default
```

Start network:

```
simulaqron start
```

Expected:

```
Network running
```

## 11 Step 6: Install ngrok

On BOTH IITR and IITJ:

Go to:

<https://ngrok.com/download>

OR:

```
snap install ngrok
```

## 12 Step 7: Create ngrok Account

Go to:

<https://dashboard.ngrok.com>

Copy your authentication token.

## 13 Step 8: Authenticate ngrok

On BOTH systems:

```
ngrok config add-authtoken YOUR_TOKEN
```

## 14 Step 9: Configure IITR

On IITR:

Open:

```
config.py
```

Update:

```
NODE_ID = "IITR"
```

```
NODE_ROLE = "SERVER"
```

```
HOST = "0.0.0.0"
```

```
PORT = 8000
```

## 15 Step 10: Configure IITJ

On IITJ:

Open:

```
config.py
```

Update:

```
NODE_ID = "IITJ"
```

```
NODE_ROLE = "CLIENT"
```

```
HOST = "0.0.0.0"
```

```
PORT = 8001
```

## 16 Step 11: Start SimulaQron

On BOTH IITR and IITJ:

```
cd ~/qkd_demo
```

```
source simulaqron_env/bin/activate
```

```
simulaqron start
```

## 17 What Happens Internally in SimulaQron

SimulaQron creates:

- virtual quantum nodes
- simulated qubits
- quantum channels
- measurement operations

It simulates the BB84 protocol behavior locally. :contentReference[oaicite:1]index=1

## 18 Step 12: Start IITR KMS Server

On IITR:

```
cd ~/qkd-network-emulation/May11th
```

```
source ~/qkd_demo/simulaqron_env/bin/activate
```

```
python kms_server.py
```

Expected:

```
[IITR BB84 KEY] id=0  
[IITR BB84 KEY] id=1  
...
```

## 19 What Happens Internally at IITR

### 19.1 Quantum Layer

The QKDManager:

- generates random bits
- generates random bases
- creates qubits
- applies Hadamard gates
- performs BB84 encoding

using SimulaQron.

### 19.2 Basis Reconciliation

Only matching bases are retained.

This produces the sifted key.

### 19.3 QBER

QBER is computed:

$$QBER = \frac{Errors}{Compared Bits}$$

### 19.4 AES Key Derivation

The sifted bits are converted into AES-256 keys using SHA-256.

## 20 Step 13: Start ngrok on IITR

Open NEW terminal on IITR:

```
ngrok http 8000
```

Example output:

```
https://iitr-demo.ngrok-free.app
```

Copy this URL.

## 21 Step 14: Start IITJ KMS Server

On IITJ:

```
cd ~/qkd-network-emulation/May11th
```

```
source ~/qkd_demo/simulaqron_env/bin/activate
```

```
python kms_server.py
```

Expected:

```
[IITJ] Waiting for BB84 synchronization
```

## 22 What Happens Internally at IITJ

The InterKMSSClient:

- contacts IITR
- synchronizes metadata
- validates hashes
- validates synchronization index
- verifies replay protection

## 23 Step 15: Start ngrok on IITJ

Open NEW terminal:

```
ngrok http 8001
```

Example:

```
https://iitj-demo.ngrok-free.app
```

Copy this URL.

## 24 Step 16: Update PEER\_NODES

On BOTH IITR and IITJ:

Open:

```
config.py
```

Update:

```
PEER_NODES = {  
    "IITR":  
        "https://iitr-demo.ngrok-free.app",  
    "IITJ":  
        "https://iitj-demo.ngrok-free.app"  
}
```

## 25 Step 17: Restart Both Servers

Stop:

CTRL + C

Restart on BOTH systems:

```
python kms_server.py
```

## 26 Step 18: Send Secure Message

On IITR:

```
python send_message.py
```

Enter:

```
Hello IITJ from IITR using BB84 QKD
```

## 27 What Happens During Communication

### 27.1 1. Key Retrieval

Sender fetches synchronized BB84-derived AES key.

### 27.2 2. Metadata Generation

MetadataManager generates:

- session ID
- nonce
- timestamp
- sync index
- SHA-256 hash

### 27.3 3. AES Encryption

Message encrypted using:

*AES – 256 – GCM*

### 27.4 4. Synchronization

InterKMSSClient synchronizes:

- metadata
- hash
- synchronization index

### 27.5 5. Internet Transfer

Ciphertext transferred through:

IITR -> ngrok -> Internet -> ngrok -> IITJ

### 27.6 6. Replay Protection

Receiver validates:

- nonce
- message ID
- timestamp

### 27.7 7. Hash Verification

Receiver verifies SHA-256 synchronization hash.

### 27.8 8. AES Decryption

Receiver decrypts ciphertext.

## 28 Expected IITR Output

[ENCRYPTION SUCCESS]

Synchronization:  
VERIFIED

Delivery:  
SUCCESS

## 29 Expected IITJ Output

RECEIVED SECURE MESSAGE

Protocol:

BB84

Synchronization:

VERIFIED

Replay Protection:

PASSED

Message:

Hello IITJ from IITR using BB84 QKD

## 30 How Key Synchronization Works

### 30.1 Important

Raw quantum bits are NOT transmitted through the internet.

Instead:

- IITR generates BB84-derived key
- metadata generated
- synchronization hash exchanged
- IITJ validates synchronization

### 30.2 Why This Is Important

Real QKD systems never expose raw quantum states publicly.

Only metadata and verification information travel over the classical channel.

Your architecture follows this hybrid-QKD design principle. :contentReference[oaicite:2]index=2

## 31 Quantum vs Classical Channels

|                      |                                     |
|----------------------|-------------------------------------|
| Quantum Channel      | Simulated locally using SimulaQron  |
| Classical Channel    | Real internet using ngrok + FastAPI |
| Quantum Operations   | Qubits, Hadamard, measurements      |
| Classical Operations | Metadata, ciphertext, hashes        |
| Synchronization      | Real distributed synchronization    |

## 32 Research Significance

This architecture is no longer a simple BB84 simulator.

It becomes:

### **Distributed Hybrid Quantum-Classical Secure Communication Framework**

with:

- distributed KMS
- BB84 integration
- ETSI APIs
- AES-GCM
- replay protection
- metadata synchronization
- distributed orchestration
- internet-based communication

The architecture aligns closely with modern distributed QKD orchestration frameworks and emulated QKD deployments. :contentReference[oaicite:3]index=3